

Assignment Number 1

Group 17:

Darlington Nkrumah (202492437)

Greg de Souza (202582225)

Xuan Toan Doan (202583882)

February 2026

Introduction

The assignment provides data on the glucose metabolism measurements taken from isthmuscingulate and precuneus regions on the brain as a possible measurement to diagnose dementia of Alzheimer's type (DAT), labeled sDAT/1. The data on healthy individuals (NC) is labeled sNC/0 on the report. The data was proved in two groups - The training data, and the test data, used in their respective functions.

With this data, we implemented studied the implementation of a Nearest Neighbor classifier using various number of neighbors (k) and metrics to evaluate the performance in the context of over/under fitting, bias and variance.

The code was implemented using the python library *scikit-learn 1.8.0* for the classifier and accuracy scores, the *pandas 2.3.3* for data handling, and *matplotlib 3.10.8* for plotting.

Question 1

The first group Nearest Neighbors classifiers (refereed from now on as kNN) uses the *Euclidean* for values of $k=1, 3, 5, 10, 20, 30, 50, 100, 150, 200$. The resulted boundary regions overlaid with the training and testing data is presented on figures 1 to 9, the training and test error for each value of k is presented on the table 1.

The **best performance** for this metric is obtained for $k = 30$ neighbors, with a testing error of 0.170, beating the performance of the other kNN classifiers by at least 5%. Other **intermediate values** of k ($k = 5, 10$ and 20) also performed well when in comparison to the best performing model ($k = 30$).

For the **low values** of k , the testing error was 25% higher for $k = 3$, and 37.5% higher for $k=1$, even though the training error was 0. This is the result of **over fitting**, as the capacity of the kNN classifier **increases** as k **decreases**. This is specially notable for the $k=1$ case, where each point is its own closest neighbor, as a result

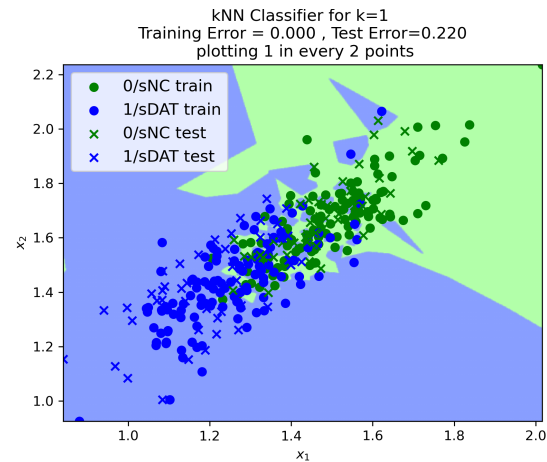


Figure 1: Nearest Neighbor classifier for $k = 1$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

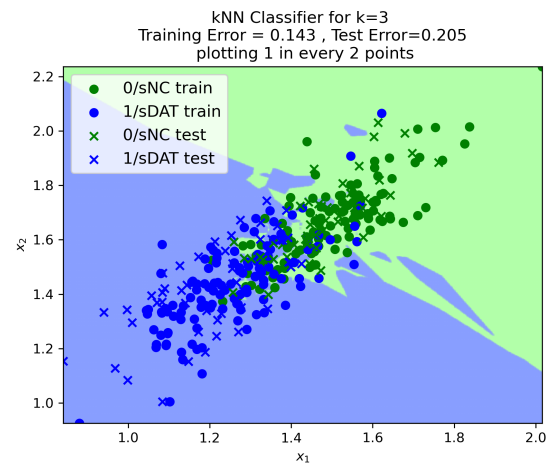


Figure 2: Nearest Neighbor classifier for $k = 3$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

k	Training Error	Testing Error
1	0	0.220
3	0.143	0.205
5	0.156	0.170
10	0.152	0.170
20	0.169	0.170
30	0.169	0.160
50	0.158	0.190
100	0.196	0.200
150	0.192	0.190
200	0.222	0.205

Table 1: Training and error rate of kNN classifiers for $k = 1, 3, 5, 10, 20, 30, 50, 100, 150, 200$

it can fit any dataset with training error of 0, even if the generalized error is high, indicating that low values of k are unable to perform good predictions on unseen data. In term of bias and variance, low values of k have **low bias**, as they can easily conform to any data set, while having **high variance**, as different datasets with generate significantly different models.

For the **high values** of k , the testing error was also over 20% worst then the best performing classifier, but for different reasons than the low value case. For high values of k , the model capacity **decreases**. For example, in the extreme where k is equal in size to the dataset, a kNN classifier will classify any point was the majority class, due to it counting every point as a neighbor.

Even before this extreme scenario, it's possible to see the increase in bias of the model as k increases. For low values of k , the boundary regions are highly non-linear and discontinuous, this also represents well the high variance of low k kNN classifiers, as a different dataset would result in a significantly different boundary region. As k increases, the boundary between the sNC and sDAT regions becomes more "well-behaved". In intermediate values of k , the non-linear behavior is enough to differentiates classes in the region where the sNC and sDAT data set mixes, but not too much to over fit the training data. And for high values k , the bias is too high, and the boundary becomes basically a straight line separating the regions with the majority of each class, so even a different dataset would still generate a similar boundary region, as the kNN classifier is low variance for high values of k .

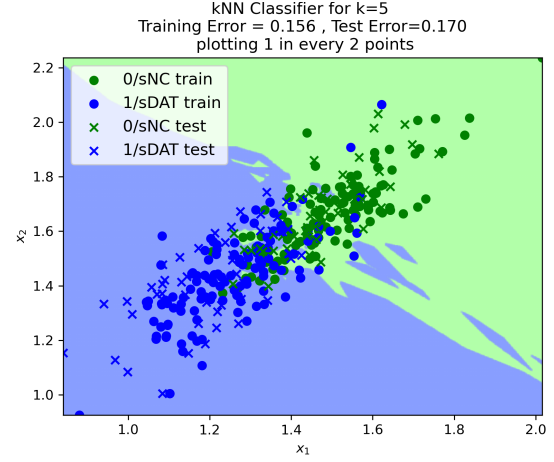


Figure 3: Nearest Neighbor classifier for $k = 5$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

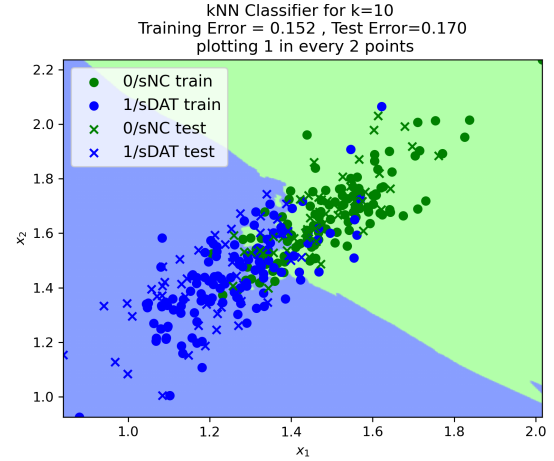


Figure 4: Nearest Neighbor classifier for $k = 10$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

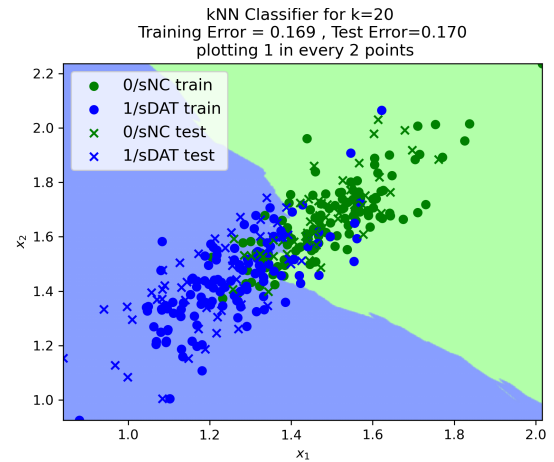


Figure 5: Nearest Neighbor classifier for $k = 20$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

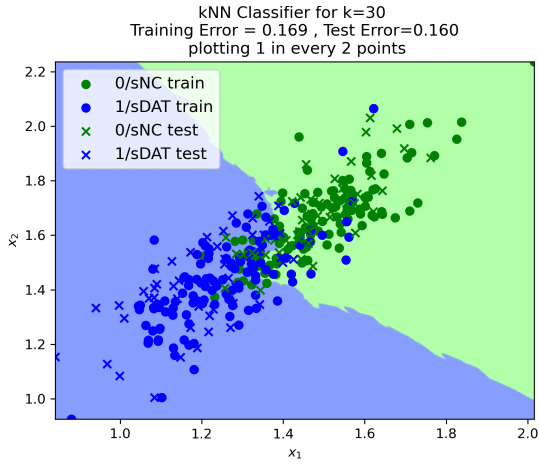


Figure 6: Nearest Neighbor classifier for $k = 30$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

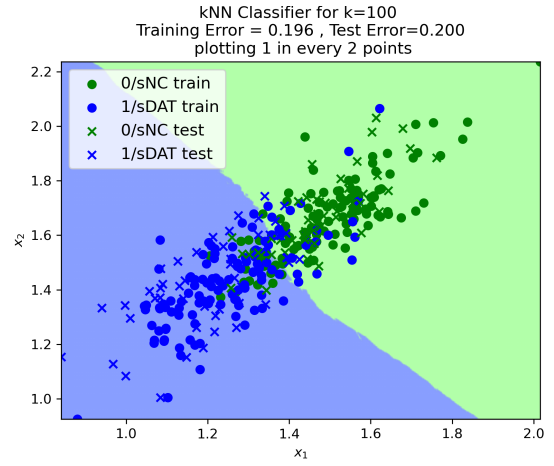


Figure 7: Nearest Neighbor classifier for $k = 100$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

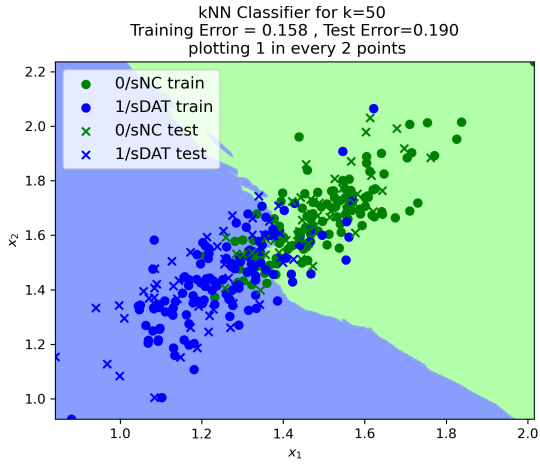


Figure 8: Nearest Neighbor classifier for $k = 50$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

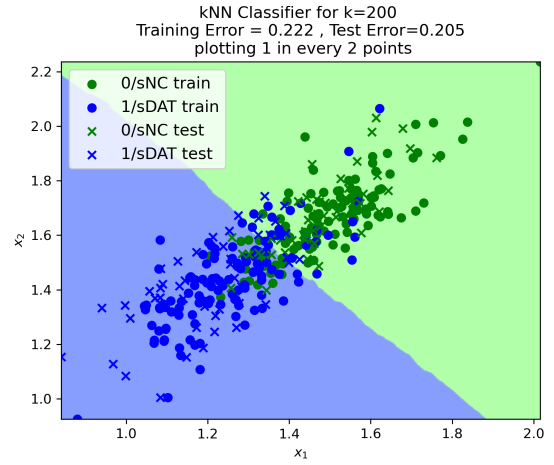


Figure 9: Nearest Neighbor classifier for $k = 200$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

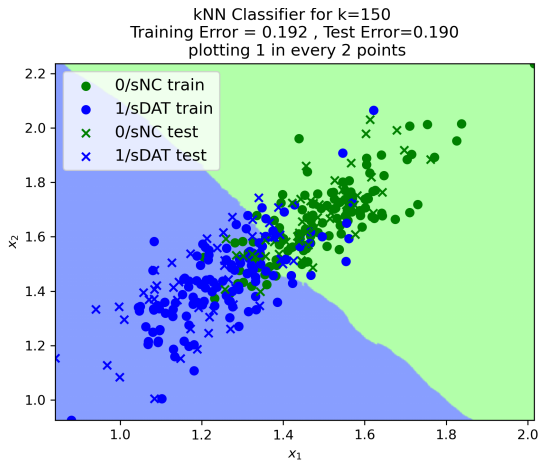


Figure 10: Nearest Neighbor classifier for $k = 150$ with sDAT region in blue and sNC region colored green, with training and test data overlayed

Question 2

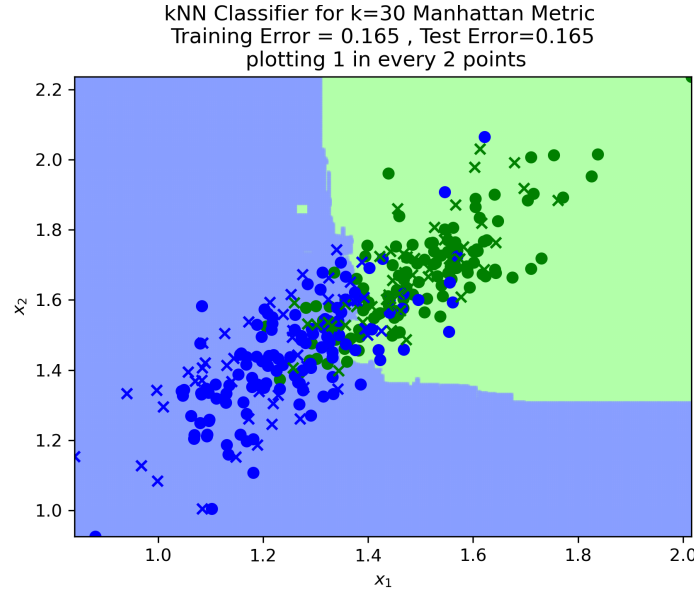


Figure 11: kNN classifier for k=30 using the **Manhattan** metric, sNC region in green, sDAT region in blue

As shown in the table 1, k=30 was the best performing kNN classifier for the *Euclidean* metric. We now compare this classifier with the equivalent classifier using the *Manhattan* metric.

Since the each feature vector has two features, the euclidean distance between two features ($\bar{x} = (x_1, x_2)$ and $\bar{y} = (y_1, y_2)$) is given by equation 1, and the Manhattan Distance is given by equation 2. The boundary visualization graph for the Manhattan metric is provided in figure 4. The “blocky” nature of the Manhattan metric is visible in the boundary division between the two evaluated classes (sNC and sDAT).

$$D_{Euclidean} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2} \quad (1)$$

$$D_{Manhattan} = |x_1 - y_1| + |x_2 - y_2| \quad (2)$$

The *Euclidean* metric resulted in a training error of $err_{Euclidean} = 0.169$, and a testing error over the given test data set is $Err(D)_{Euclidean} = 0.160$. For the *Manhattan* metric, the training error is $err_{Manhattan} = 0.165$, and the testing error is over the same data set is $Err(D)_{Manhattan} = 0.165$. The **Euclidean** metric has a **better performance** than the Manhattan metric in this given test data set. This suggests that the kNN classifier generalizes better with the Manhattan metric, specially when we consider that the **training error** was lower for the Manhattan metric, even though it’s generalized error was worse.

Question 3

In the last section we concluded that the Euclidean metric outperformed the Manhattan metric for the kNN classifier in the provided datasets. For k=30, the best performer of question 1, the Manhattan metric had a generalized error of 0.165, worse than the generalized error for the Euclidean metric (0.160).

We now evaluate the training and testing error of the kNN classifier with Euclidean metric versus it’s model capacity, modeled as $capacity = 1/k$. For that, we trained the model from k=1 to k=100 (inclusive), generating the capacity between 0.1 and 1. The results are shown in figure 5.

The most notable feature of the error rate evolution, is the training error going to 0 at high model capacity, even though the testing/generalized error is higher than for lower model capacities. This is a visualization of the **over-fitting** problem that can affect high capacity model, where the training error fails to represent the generalization error due to high variance.

Reflecting the discussion of question 1, the low capacity models are also unable to generalize well, but due to excessive bias, resulting in higher testing error than models with average capacity. The lowest training error

is shown by the dotted line, and it happens for $k=30$, suggesting that the best kNN classifier from the options evaluated until now is indeed the kNN for $k=30$ and the *Euclidean* metric.

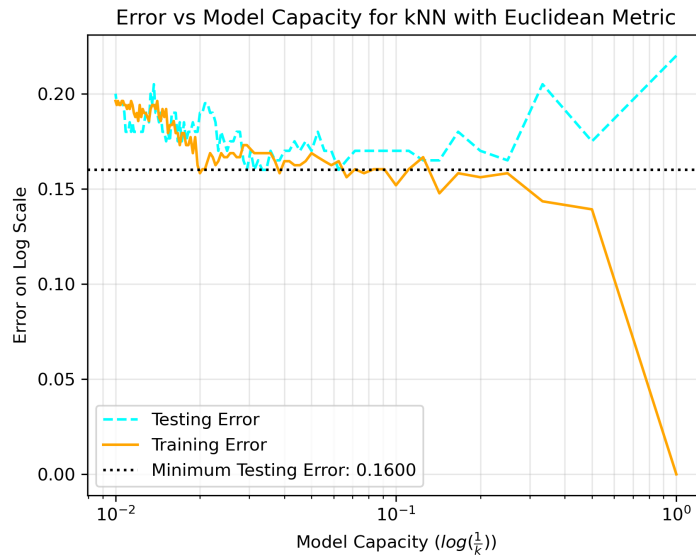


Figure 12: Error rate for kNN classifier with Euclidean Metric versus model capacity (1), where k is the number of neighbors. The capacity is shown in logscale